

COMP9312 Project — Q1

First Cycle-Causing Edge in an Insertion Stream

YUHUI LUO (z5565446)

1. Problem

We start from n isolated vertices $V = \{0, 1, \dots, n-1\}$ and an *undirected, unweighted* edge stream $L = [(u_1, v_1), (u_2, v_2), \dots]$, and insert the edges one at a time. We must return the **first** inserted edge that closes a cycle together with **one** cycle containing it, or **None** if the stream stays acyclic. We may assume $0 \leq v < n$ for every endpoint and that no edge is repeated.

2. Key Observation

Before any cycle appears, the accepted edges form a **forest**: each kept edge joins two vertices from *different* trees and merges them. Hence an incoming edge (u, v) creates a cycle

iff u and v already lie in the same tree,

because that tree already contains a unique $u-v$ path and the new edge closes it into a cycle. The task therefore reduces to *maintaining connected components under edge insertions* and, for each new edge, testing whether its endpoints are already connected. This is precisely what a **Disjoint Set Union** (DSU / union-find) supports.

3. Algorithm

Data structures.

- `parent[]`, `rank[]` — DSU with *path compression* and *union by rank*.
- `forest[]` — adjacency lists of the accepted (non-cycle) edges, i.e. the current spanning forest. It never holds more than $n-1$ edges.

Processing the stream. For each edge (u, v) in order:

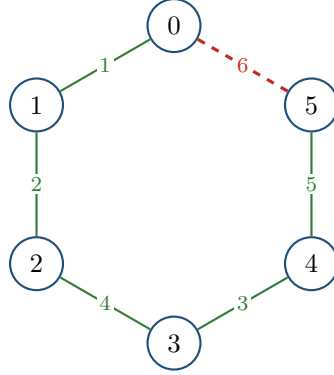
1. If `find(u) = find(v)`, then u, v are already connected: (u, v) is the first cycle-causing edge. Recover the cycle and return.
2. Otherwise accept it: append (u, v) to `forest`, call `union(u, v)`, and continue.

If the loop ends with no collision, return **None**.

Recovering the cycle. When the collision edge (u, v) is found, u and v are in the *same* tree of the spanning forest. A single **BFS from** u inside `forest` finds the unique tree path $u \rightarrow \dots \rightarrow v$; appending u closes the cycle. We return $[[u, v], [u, \dots, v, u]]$.

3.1 Worked Example (spec Example 1)

$n = 6$, $L = [(0, 1), (1, 2), (3, 4), (2, 3), (4, 5), (5, 0)]$. The first five edges are accepted and build the single tree $0-1-2-3-4-5$. The sixth edge $(5, 0)$ satisfies $\text{find}(5) = \text{find}(0)$ and is the first cycle edge; BFS from 5 reaches 0 along $5-4-3-2-1-0$.



■ accepted forest edges (labelled by insertion order)

■ edge 6 = $(5, 0)$ closes the cycle

Result. $[[5, 0], [5, 4, 3, 2, 1, 0, 5]]$

4. Correctness

- **First cycle detection.** By induction the accepted edges are always a forest, so DSU components are exactly the trees. The test $\text{find}(u) = \text{find}(v)$ is true *iff* a $u-v$ path already exists, i.e. *iff* (u, v) would form a cycle. Edges are examined in stream order and we return on the first collision, so it is *the* first cycle-causing edge.
- **Valid cycle.** At the collision the forest is not yet modified by (u, v) , so the BFS path $u \rightarrow \dots \rightarrow v$ consists only of genuine earlier edges; with (u, v) it forms a simple cycle containing the new edge.

5. Complexity Analysis

Let n be the number of vertices and $m = |L|$ the number of inserted edges.

5.1 Time complexity: $O(m \cdot \alpha(n) + n)$

Each edge performs two find operations and at most one union . With path compression and union by rank, each DSU operation costs amortised $O(\alpha(n))$, where α is the inverse Ackermann function ($\alpha(n) \leq 4$ for all practical n). Over the whole stream this is $O(m \cdot \alpha(n))$. The cycle-recovery BFS runs *once*, over a forest of at most $n - 1$ edges, costing $O(n)$. Hence the total query time is

$$O(m \cdot \alpha(n) + n),$$

which is effectively linear in the size of the input.

5.2 Space complexity: $O(n)$

- `parent[]` and `rank[]`: $O(n)$.
- `forest[]`: stores only accepted edges; a forest on n vertices has at most $n - 1$ edges, so $O(n)$ — *independent of m* .

- BFS prev map and queue: $O(n)$.

Total working space is $O(n)$. In particular we never store the whole edge set, only the growing spanning forest, so the data structure needed for query processing is $O(n)$ rather than $O(n + m)$.

6. Reference Implementation

```
class FirstCycleEdgeQuery:
    def query(self, n, L):
        parent = list(range(n)); rank = [0] * n
        tree = [[] for _ in range(n)] # accepted edges = spanning forest

        def find(x):
            root = x
            while parent[root] != root: root = parent[root]
            while parent[x] != root: parent[x], x = root, parent[x] # compress
            return root

        for u, v in L:
            ru, rv = find(u), find(v)
            if ru == rv: # u, v already connected -> cycle
                return [[u, v], self._cycle(tree, u, v)]
            tree[u].append(v); tree[v].append(u)
            if rank[ru] < rank[rv]: ru, rv = rv, ru
            parent[rv] = ru
            if rank[ru] == rank[rv]: rank[ru] += 1
        return None

    def _cycle(self, tree, u, v): # BFS the tree path u..v, close it
        prev = {u: u}; q = deque([u])
        while q:
            x = q.popleft()
            if x == v: break
            for y in tree[x]:
                if y not in prev: prev[y] = x; q.append(y)
        path = [v]
        while path[-1] != u: path.append(prev[path[-1]])
        path.reverse(); path.append(u) # [u, ..., v, u]
        return path
```